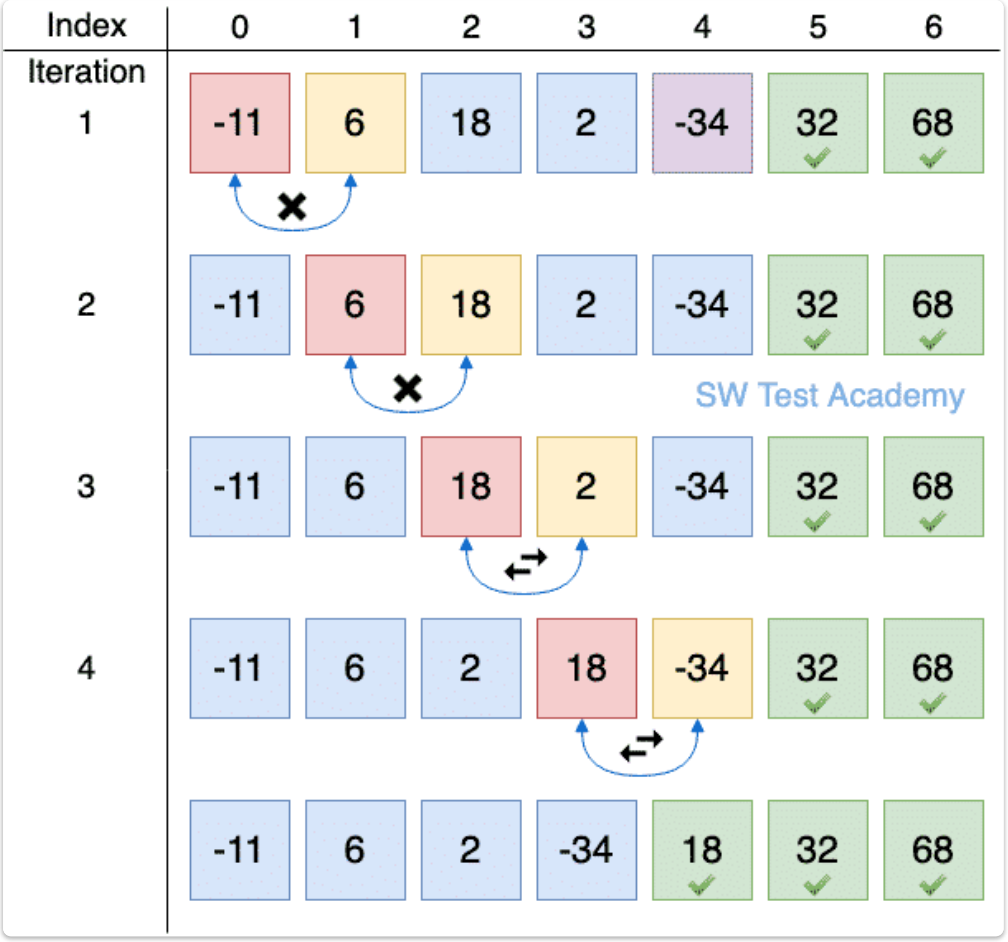
Bubble-Sort

Bubble Sort

Bubble Sort (ordenamiento burbuja) es un algoritmo de ordenación muy sencillo que:

- Compara elementos adyacentes en el array.
- Si están en el orden incorrecto, los intercambia (`swapped`) .
- En cada pasada completa, el elemento más grande "sube" al final del array, como una burbuja que asciende.

Este proceso se repite hasta que ya no se realiza ningún intercambio en una pasada, lo que significa que el array está ordenado.



Propiedades clave:

- In-place:
  No usa estructuras auxiliares grandes, solo unas pocas variables.

- Estable:
  No cambia el orden relativo de elementos con el mismo valor (cuando solo intercambiamos si `A[i] > A[i+1]`).
- Complejidad temporal (versión con bandera `swapped`):
  - Mejor caso (array ya ordenado): (O(n))
  - Caso medio: (O(n^2))
  - Peor caso (orden inverso): (O(n^2))
- Complejidad espacial:
  - (O(1)) espacio extra.
- Uso típico:
  - Didáctico para entender comparación + intercambio.
  - En la práctica, se usa solo para listas muy pequeñas o contextos educativos.

Esquema Bubble-Sort

Tenemos un array `A` de longitud `n`.
La idea de Bubble Sort es:

- Repetir pasadas por el array.
- En cada pasada, comparar pares adyacentes `A[i]` y `A[i+1]`.
- Si están desordenados (`A[i] > A[i+1]`), se intercambian.
- Con cada pasada, el máximo del segmento no ordenado acaba en la última posición de ese segmento.
- Usar una bandera (`swapped`) para saber si hubo algún intercambio. Si en una pasada completa no hay intercambios, ya está ordenado y podemos terminar antes de llegar al peor caso.

Pasos más detallados:

1. Inicializar el rango activo y la bandera
   - `n = length(A)` → tamaño actual del segmento no ordenado.
   - `swapped = true` → para entrar en el bucle principal.
2. Repetir mientras haya intercambios (`swapped == true`):

- Al inicio de cada pasada:
  - Poner `swapped = false`.
- Recorrer el array desde el índice `0` hasta `n - 2`:
  1. Mirar la pareja `(A[i], A[i+1])`.
  2. Si `A[i] > A[i+1]`:
     - Intercambiar `A[i]` y `A[i+1]`.
     - Marcar `swapped = true` (se ha realizado al menos un intercambio).

3. **Reducir el rango de comparación**
   - Tras cada pasada, el elemento más grande del segmento actual ha quedado en `A[n-1]`.
   - Podemos reducir el rango: `n = n - 1`, porque `A[n-1]` ya está en su posición definitiva.
4. **Condición de parada**
   - Si terminamos una pasada y `swapped` sigue siendo `false`, significa que no se ha movido nada: **el array ya está ordenado**.
   - En ese momento salimos del bucle.

Mini ejemplo rápido

Para `A = [5, 3, 8, 4, 2]`:

- 1ª pasada: comparamos (5,3) → swap, (5,8), (8,4) → swap, (8,2) → swap
  Resultado: `[3, 5, 4, 2, 8]` (8 ya está en su sitio al final)
- 2ª pasada (sin tocar el último): comparamos (3,5), (5,4) → swap, (5,2) → swap
  Resultado: `[3, 4, 2, 5, 8]`
- ...
- Cuando una pasada no produce intercambios, terminamos.

Pseudocódigo Bubble-Sort

```
BUBBLE-SORT(A)
    n = length(A)
```

```
    swapped = true

    while swapped
        swapped = false

        for i = 0 to n - 2
            if A[i] > A[i + 1]
                temp = A[i]
                A[i] = A[i + 1]
                A[i + 1] = temp

                swapped = true

        n = n - 1
```

Implementación en Python

```python
def bubble_sort(arr):
    n = len(arr)
    swapped = True

    # Repetimos mientras haya habido algún intercambio
    while swapped:
        swapped = False

        # Recorremos el array hasta n - 1 (comparando i e i+1)
        for i in range(n - 1):
            if arr[i] > arr[i + 1]:
                # Intercambiamos si están en el orden incorrecto
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                swapped = True

        # El último elemento ya está en su posición correcta
        n -= 1


if __name__ == "__main__":
```

```
unsorted_array = [5, 3, 8, 4, 2]
bubble_sort(unsorted_array)
print(unsorted_array)  # [2, 3, 4, 5, 8]
```